Unit 1.0 — Fundamentals of WWW and HTML

Fundamentals of World Wide Web

Web Browser: A software application (e.g., Chrome, Firefox, Edge) used to access, retrieve and display content on the World Wide Web by interpreting HTML, CSS and JavaScript.

URL (Uniform Resource Locator): The address used to locate a resource on the internet. Example: `https://www.example.com/page.html`

Web page: A single document on the web written in HTML, displayed by a browser.

Website: A collection of related web pages, linked together and identified by a common domain name.

Types of sites: - **Static website:** Content is fixed; same content is displayed to every user; written purely in HTML/CSS; does not interact with a database. - **Dynamic website:** Content changes based on user interaction, time, or database; uses server-side scripting (PHP, ASP.NET, JSP) along with HTML/CSS/JS.

Client-server model: The client (browser) sends an HTTP request to the server; the server processes the request and sends back an HTTP response (web page) to be rendered by the client.

Web Servers: Software (e.g., Apache, IIS, Nginx) that stores, processes and delivers web pages to clients upon request.

Website Hosting: The process of storing website files on a server so that it is accessible over the internet/intranet.

Internet vs Intranet: - **Internet:** A global, public network connecting millions of computers worldwide. - **Intranet:** A private network restricted to an organization, using the same technology as the internet.

Publishing website on Intranet / Installing and configuring a web server: Install server software (e.g., Apache/XAMPP), place website files in the server's root directory (e.g., `htdocs`), start the server service, and access the site using `localhost` or the server's IP address within the local network.

1.1 HTML

HTML (HyperText Markup Language) is a markup language used to define the structure and content of web pages using elements represented by tags.

Web page structure:

```
<!DOCTYPE html>
<html>
<head>
    <title>My First Page</title>
    <meta charset="UTF-8">
    <meta name="description" content="Sample page">
</head>
<body>
    <h1>Welcome</h1>
    <p>This is a paragraph.</p>
</body>
</html>
```

- **Tags:** Keywords enclosed in angle brackets, e.g., `<p>`, `<h1>`. Most tags have an opening and a closing tag (`<p>...</p>`); some are empty/void elements (`<br>`, `<img>`).
- **Attributes:** Provide additional information about an element, written inside the opening tag. Example: `<img src="pic.jpg" alt="photo">`
- **DOCTYPE:** Declares the HTML version being used; `<!DOCTYPE html>` declares HTML5.
- **`<head>`:** Contains meta-information not displayed on the page (title, meta tags, links to CSS).
- **`<body>`:** Contains the visible content of the web page.
- **`<title>`:** Sets the title shown on the browser tab.
- **Meta tags:** Provide metadata such as character set, author, description, keywords for SEO.

```
<meta name="author" content="John Doe">
<meta name="keywords" content="HTML, CSS, JS">
```

1.2 Block Level Tags

Block-level elements start on a new line and occupy the full available width.

```
<h1>Heading 1</h1> ... <h6>Heading 6</h6>   <!-- Headings -->
<p>This is a paragraph.</p>                  <!-- Paragraph -->
Line one<br>Line two                         <!-- Line break -->
<div>Division block</div>                    <!-- Division -->
<center>Centered text</center>               <!-- Centered text (deprecated, use CSS) -->
<blockquote>Quoted text from a source</blockquote>  <!-- Block Quote -->
<pre>   Preformatted
   text with spacing</pre>                   <!-- Preformatted text -->
<hr>                                           <!-- Horizontal rule/line -->
```

1.3 Text Level Tags and Special Characters

Text-level (inline) tags format a portion of text without breaking the line flow.

```
<b>Bold text</b>            <strong>Important text</strong>
<i>Italic text</i>          <em>Emphasized text</em>
<u>Underlined text</u>
<strike>Strikethrough text</strike>  or  <s>Strikethrough</s>
<sup>Superscript</sup>      <sub>Subscript</sub>
<div>A generic container for styling/grouping</div>
```

Displaying special characters (HTML entities):

```
&lt;   &gt;   &amp;   &copy;    
```

Example: `&lt;p&gt;` displays as `<p>` on the rendered page.

1.4 Lists

Ordered List:

```
<ol>
  <li>Step One</li>
  <li>Step Two</li>
</ol>
```

Unordered List:

```
<ul>
  <li>Apple</li>
  <li>Banana</li>
</ul>
```

Definition List:

```
<dl>
  <dt>HTML</dt>
  <dd>HyperText Markup Language</dd>
</dl>
```

Nested Lists:

```
<ul>
  <li>Fruits
    <ul>
      <li>Apple</li>
      <li>Mango</li>
    </ul>
  </li>
</ul>
```

Unit 2.0 — URL, Image, Table and Frame in HTML

2.1 URL and Anchor Tag

Types of URL: - **Absolute URL:** Contains the full path including protocol and domain. Example: `https://www.site.com/images/pic.jpg` - **Relative URL:** Path relative to the current document's location. Example: `images/pic.jpg`

Pros and cons: - Absolute URL — works from anywhere but breaks if the domain changes. - Relative URL — easier to maintain within a site, but does not work if files are moved outside the site structure.

Anchor tag (`<a>`): Used to create hyperlinks.

```
        <a href="https://www.google.com">Visit Google</a>          <!-- external link -->
        <a href="about.html">About Us</a>                          <!-- internal link -->
        <a href="#section2">Go to Section 2</a>                    <!-- link to area in same page -->
        <h2 id="section2">Section 2</h2>
        <a href="page2.html#top">Go to top of page2</a>             <!-- link to area in another page -->
        <a href="mailto:abc@example.com">Email Us</a>               <!-- email link -->
```

2.2 Images, Colors and Backgrounds

Inserting images:

```
        <img src="photo.jpg" alt="A photo" width="300" height="200" border="2" align="left">
```

- `src` — image source path
- `alt` — alternate text if image fails to load
- `width` / `height` — sizing
- `align` — alignment (left/right/center)
- `border` — border thickness

Image as page background:

```
        <body background="bg.jpg">
```

Solid color page background:

```
        <body bgcolor="lightblue">
        <!-- or using CSS -->
        <body style="background-color:lightblue;">
```

2.3 Table

```
        <table border="1" cellspacing="5" cellpadding="10" width="50%" align="center" bgcolor="#f0f0f0">
          <tr>
            <th>Name</th><th>Marks</th>
          </tr>
          <tr>
            <td>Aman</td><td>89</td>
          </tr>
        </table>
```

- `<table>` — defines the table
- `<tr>` — table row
- `<th>` — table header cell (bold, centered by default)
- `<td>` — table data cell
- **Attributes:** `border` (border thickness), `cellspacing` (space between cells), `cellpadding` (space inside a cell), `width`, `align`, `bgcolor`

2.4 Frames

Frames allow a browser window to be divided into multiple independent sections, each displaying a separate HTML document.

Types of Frames: Row-wise split, column-wise split, and nested combination of both.

```
        <frameset cols="30%,70%">
           <frame src="menu.html">
           <frame src="content.html">
        </frameset>
```

```
        <frameset rows="20%,80%">
           <frame src="header.html">
           <frame src="body.html">
        </frameset>
```

- `cols` attribute — splits the window into vertical frames.
- `rows` attribute — splits the window into horizontal frames. (Note: `<frame>` / `<frameset>` are deprecated in HTML5; `<iframe>` is the modern replacement for embedding a document within a page.)

```
        <iframe src="page.html" width="400" height="300"></iframe>
```

Unit 3.0 — Cascading Style Sheets (CSS)

3.1 Basics of CSS

CSS (Cascading Style Sheets) is a style sheet language used to describe the presentation (look and formatting) of a document written in HTML.

Benefits of CSS: Separation of content from design, consistency across pages, reduced code repetition, easier maintenance, faster page loading, responsive design support.

Types of Style Sheets / Adding style to a document:

1. **Inline CSS** — applied directly to a single HTML element using the `style` attribute.

```
<p style="color:red; font-size:18px;">Inline styled text</p>
```

2. **Internal/Embedded CSS** — written inside a `<style>` tag in the `<head>` section.

```
<head>
<style>
  p { color: blue; font-size: 16px; }
</style>
</head>
```

3. **External CSS** — written in a separate `.css` file and linked using `<link>`.

```
<head>
<link rel="stylesheet" type="text/css" href="style.css">
</head>
```

```
/* style.css */
p { color: green; }
```

Selectors: - **CLASS selector:** Applies a style to any element having that class (reusable).

```
<p class="highlight">Text</p>
```

```
.highlight { background-color: yellow; }
```

- **ID selector:** Applies a style to a single unique element.

```
<p id="header-text">Text</p>
```

```
#header-text { font-weight: bold; }
```

3.2 Style Sheet Properties

Font and Text properties:

```
p {
  font-family: Arial, sans-serif;
  font-size: 14px;
  font-weight: bold;
  font-style: italic;
  text-align: center;
  text-decoration: underline;
  line-height: 1.5;
}
```

Box properties:

```
div {
  width: 200px;
  height: 100px;
  margin: 10px;
  padding: 5px;
  border: 1px solid black;
}
```

Color and background properties:

```
body {
  color: #333333;                /* text color */
  background-color: #f0f0f0;     /* background color */
  background-image: url("bg.png");
}
```

Background and color gradients in CSS:

```
div {
  background: linear-gradient(to right, red, yellow);
}
```

Creating and using an external CSS file: Write all rules in `style.css` and link it in every HTML page using `<link>` — promotes reuse across multiple pages.

Using internal and inline CSS: Useful for page-specific or element-specific quick styling, though external CSS is preferred for larger projects.

Setting font/text using table layout:

```
table { font-family: Verdana; font-size: 12px; }
th { background-color: navy; color: white; }
```

Unit 4.0 — Basic JavaScript

4.1 JavaScript Introduction

JavaScript is a lightweight, interpreted, client-side scripting language used to make web pages interactive and dynamic (DHTML).

Writing and executing JavaScript:

```
<script>
  document.write("Hello, JavaScript!");
</script>
```

JavaScript can be written internally (inside `<script>` tag), externally (linked `.js` file), or inline (in an event attribute).

Variables:

```
var x = 10;     // function/global scope, older style
let y = 20;     // block scope, modern
const z = 30;   // constant, cannot be reassigned
```

Data types: Number, String, Boolean, Undefined, Null, Object, Array.

```
let age = 21;             // Number
let name = "Aman";        // String
let isPass = true;        // Boolean
```

Operators: Arithmetic (`+ - * / %`), Relational (`< > == ===`), Logical (`&& || !`), Assignment (`= += -=`).

Pop-up boxes:

```
alert("This is an alert box");
let result = confirm("Are you sure?");      // returns true/false
let name = prompt("Enter your name:", "");  // returns user input as string
```

4.2 Control Structures

if-else:

```
let marks = 75;
if (marks >= 40) {
    document.write("Pass");
} else {
    document.write("Fail");
}
```

switch:

```
let day = 2;
switch(day) {
    case 1: document.write("Monday"); break;
    case 2: document.write("Tuesday"); break;
    default: document.write("Invalid");
}
```

Loops:

```
// for loop
for (let i = 1; i <= 5; i++) { document.write(i + " "); }
```

```
        // while loop
        let i = 1;
        while (i <= 5) { document.write(i + " "); i++; }

        // do...while loop
        let j = 1;
        do { document.write(j + " "); j++; } while (j <= 5);
```

break and continue:

```
        for (let i = 1; i <= 10; i++) {
            if (i == 5) break;          // exits loop
            if (i == 3) continue;       // skips iteration
            document.write(i + " ");
        }
```

Arrays:

```
        let fruits = ["Apple", "Banana", "Mango"];
        document.write(fruits[1]);          // Banana
        fruits.push("Orange");              // add element
        document.write(fruits.length);      // 4
```

Unit 5.0 — JavaScript Functions, Objects and Events

5.1 Functions

Declaring and calling functions:

```
function add(a, b) {
    return a + b;
}
let sum = add(5, 3);     // function call with parameters
document.write(sum);     // 8
```

Lifetime of JavaScript variables: - Variables declared with `var` / `let` inside a function exist only during that function's execution (local scope). - Global variables exist for the lifetime of the web page.

JavaScript Objects: A collection of key-value pairs (properties and methods).

```
let student = {
    name: "Aman",
    marks: 89,
    display: function() { document.write(this.name); }
};
student.display();
```

String Object methods:

```
let s = "Hello World";
s.length;            // 11
s.toUpperCase();      // HELLO WORLD
s.indexOf("World");   // 6
s.substring(0,5);     // Hello
```

Date Object:

```
let d = new Date();
document.write(d.getFullYear());   // current year
document.write(d.getDate());       // current date
```

Math Object:

```
Math.max(3,7);     // 7
Math.sqrt(16);      // 4
Math.random();       // random number between 0 and 1
Math.round(4.6);     // 5
```

5.2 Events and Exceptions

JavaScript Events: Actions/occurrences that JavaScript can detect and respond to.

```
<button onclick="alert('Button Clicked!')">Click Me</button>
<input type="text" onfocus="this.style.background='yellow'">
<body onload="alert('Page Loaded')">
```

Common events: `onclick`, `onmouseover`, `onmouseout`, `onload`, `onchange`, `onsubmit`, `onfocus`, `onblur`.

Error and exception handling — `try...catch...finally`:

```
try {
    let result = 10 / 0;
    if (!isFinite(result)) throw "Division error";
```

```
        document.write(result);
    } catch (err) {
        document.write("Error: " + err);
    } finally {
        document.write("Execution completed");
    }
```

`onerror()` **method:** Used to handle script errors globally on a page.

```
    window.onerror = function(message, url, line) {
        alert("Error: " + message + " at line " + line);
    };
```